PROGRAMACIÓN AVANZADA

EVALUACIÓN FINAL · PROYECTO DE SOFTWARE

Sistema de Gestión de Biblioteca

Aplicación de escritorio para administrar libros, autores, editoriales y usuarios, con base de datos relacional y operaciones CRUD validadas.

C# · .NET | Windows Forms | SQL Server | ADO.NET | CRUD | POO | Consultas parametrizadas | Arquitectura por capas | Git · GitHub

NOMBRES	Juan Romero John David Montilla
FICHA	54465
MATERIA	Programación Avanzada
DOCENTE	Verónica Castro Munar
AÑO	2026

CONTENIDO

Tabla de contenido

1. Introducción y objetivos 3

Introducción 3

Objetivo general 3

Objetivos específicos 3

2. Planteamiento del problema y alcance 3

Estado de implementación por módulo 4

3. Análisis de requerimientos 4

Requerimientos funcionales 4

Requerimientos no funcionales 4

4. Casos de uso 5

5. Arquitectura del sistema 6

6. Diagrama de clases y Programación Orientada a Objetos 7

Conceptos de POO evidenciados 7

7. Modelo entidad-relación y diccionario de datos 8

Diccionario de datos 8

8. Conexión con SQL Server y validaciones 9

Conexión 9

Validaciones y manejo de excepciones 9

9. Módulos desarrollados 10

10. Capturas de pantalla del sistema 10

11. Pruebas de funcionamiento 11

12. Control de versiones con Git y GitHub 12

Limitaciones conocidas y trabajo futuro 12

13. Conclusiones y recomendaciones 12

Conclusiones 12

Recomendaciones 12

14. Referencias bibliográficas 13

01 CAPÍTULO 1
Introducción y objetivos

Respetada profesora Verónica Castro Munar: con este documento le presentamos el trabajo final de Programación Avanzada, el **Sistema de Gestión de Biblioteca**, que desarrollamos entre los dos en C#, Windows Forms y SQL Server. El texto explica qué construimos, cómo está organizado, cómo se conecta a la base de datos y qué pruebas realizamos, y señala con claridad qué módulos quedaron completos y cuáles quedaron pendientes.

Introducción

Una institución educativa necesita administrar su biblioteca: sus libros, autores, editoriales y usuarios, y el control de los préstamos y devoluciones. Llevar esa información de forma manual produce duplicados, datos desactualizados y poca trazabilidad. La aplicación que desarrollamos centraliza la información bibliográfica en una base de datos relacional y ofrece formularios para registrar, consultar, actualizar y eliminar registros, con validaciones y mensajes claros para quien la usa.

Objetivo general

Desarrollar una aplicación de escritorio para la gestión de una biblioteca con C#, Windows Forms y SQL Server, que permita administrar libros, autores, editoriales y usuarios mediante operaciones conectadas a una base de datos relacional, aplicando los principios de la Programación Orientada a Objetos.

Objetivos específicos

OBJETIVO	CÓMO LO ABORDAMOS
Aplicar Programación Orientada a Objetos	Formularios como clases que heredan de `Form`, clase `Conexion`, encapsulamiento y constructores (capítulo 6).
Diseñar una base de datos relacional	Cinco tablas con claves primarias, foráneas y restricciones (capítulo 7).
Implementar operaciones CRUD	CRUD completo en Libros, Autores, Editoriales y Usuarios (capítulo 9).
Usar SQL Server como gestor	SQL Server LocalDB con la base `AcaBiblioteca` y consultas parametrizadas (capítulo 8).
Implementar arquitectura por capas	Arquitectura lógica de presentación, acceso a datos y base de datos (capítulo 5).
Validar datos y manejar excepciones	Campos obligatorios, tipos numéricos, confirmaciones y bloques `try/catch` (capítulos 8 y 11).
Usar Git y GitHub	Repositorio público con historial de commits (capítulo 12).
Documentar técnicamente el proyecto	Este documento, el script SQL y el `README.md`.

02 CAPÍTULO 2
Planteamiento del problema y alcance

La biblioteca de la institución gestiona cuatro catálogos que se relacionan entre sí (libros, autores, editoriales y usuarios) y un proceso operativo (préstamos y devoluciones). Los problemas que motivan el sistema son:

- La información está dispersa y sin un único punto de consulta.
- No hay control de integridad: un libro puede quedar asociado a un autor que ya no existe, o registrarse dos veces.
- No se puede saber con rapidez cuántas existencias de un libro hay disponibles.
- No existe un historial de qué usuario tiene qué libro.

Solución propuesta. Una aplicación de escritorio con un formulario principal y un menú lateral que da acceso a un formulario por módulo. Los datos se guardan en SQL Server y las relaciones entre tablas se protegen con claves foráneas, de modo que, por ejemplo, no se puede eliminar un autor que tiene libros registrados.

Estado de implementación por módulo

En la tabla siguiente indicamos con transparencia el avance real de cada módulo del alcance.

MÓDULO	FUNCIONALIDAD	ESTADO
Inicio	Pantalla de bienvenida y navegación	Implementado
Libros	Registrar, consultar, editar, eliminar y buscar por ISBN; relación con autor y editorial; existencias	Implementado
Autores	Registrar, consultar, editar y eliminar	Implementado
Editoriales	Registrar, consultar, editar y eliminar	Implementado
Usuarios	Registrar, consultar, editar y eliminar	Implementado
Préstamos y devoluciones	Formulario creado y accesible desde el menú; falta la lógica de registro, estado y devolución	Pendiente
Reportes	Formulario creado y accesible desde el menú; falta definir y programar las consultas	Pendiente

Nota. Los listados de libros, autores, editoriales y usuarios ya se consultan en la cuadrícula de cada módulo. Para los módulos pendientes dejamos propuesta la tabla `Prestamos` en el script SQL y su diseño en los capítulos 4 y 7, de modo que su implementación sea directa.

CAPÍTULO 3

03 Análisis de requerimientos

Requerimientos funcionales

ID	REQUERIMIENTO	ESTADO	EVIDENCIA
RF01	Gestión de libros: registrar, listar, editar, eliminar, buscar y relacionar con autor y editorial	Implementado	`FrmLibros`; pruebas L1–L9
RF02	Gestión de autores (identificador, nombre, apellido)	Implementado	`FrmAutores`; pruebas A1–A5
RF03	Gestión de editoriales con identificador único	Implementado	`FrmEditoriales`; pruebas E1–E6
RF04	Gestión de usuarios (nombre, apellido, documento, teléfono, correo)	Implementado	`FrmUsuarios`; pruebas U1–U5
RF05	Gestión de préstamos: usuario, libro, fechas y estado	Pendiente	`FrmPrestamos` sin lógica
RF06	Gestión de devoluciones y actualización de existencias	Pendiente	Depende de RF05
RF07	Consultas y reportes	Pendiente	`FrmReportes` sin lógica
RF08	Navegación: Inicio, Libros, Usuarios, Autores, Editoriales, Préstamos y Reportes	Implementado	`FrmPrincipal`

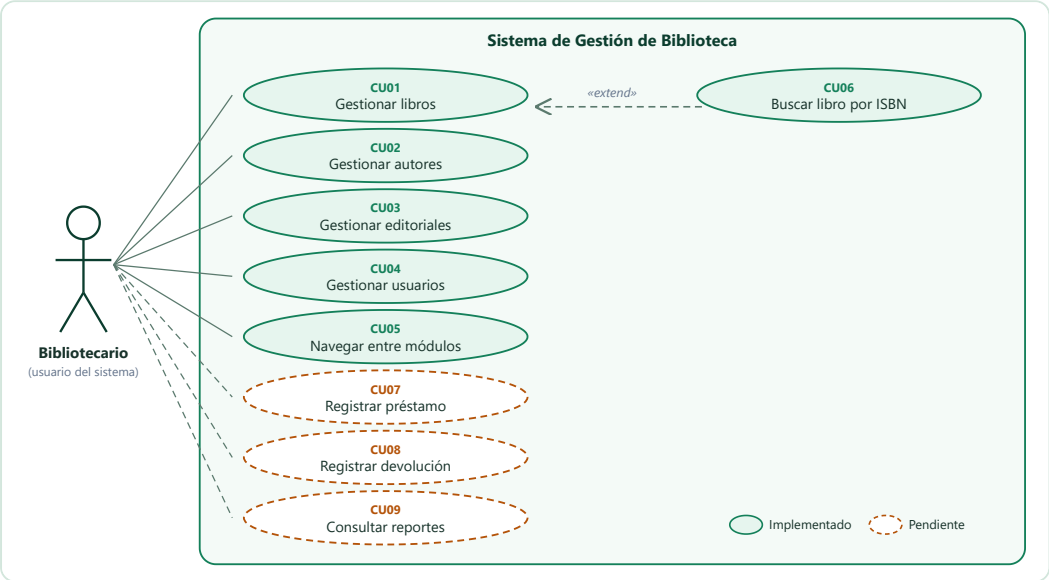
Requerimientos no funcionales

REQUERIMIENTO	ESTADO	CÓMO SE CUMPLE
Interfaz clara y organizada	Implementado	Menú lateral, panel de contenido, cuadrícula y botones con iconos
Validación de campos obligatorios	Implementado	`string.IsNullOrWhiteSpace` antes de guardar o editar
Prevención de duplicados	Implementado	Restricciones `PRIMARY KEY` y `UNIQUE`; el error se captura y se informa
Mensajes claros	Implementado	`MessageBox` de información, advertencia, error y confirmación

REQUERIMIENTO	ESTADO	CÓMO SE CUMPLE
Manejo de excepciones	Implementado	Bloques `try/catch` en cada operación contra la base
Integridad de datos	Implementado	Claves foráneas y restricciones `CHECK`
Consultas parametrizadas	Implementado	`SqlCommand` con `Parameters.AddWithValue` en las operaciones con datos del usuario
Separación de responsabilidades	Parcial	Se separa `Conexion`; el resto de la lógica está en los formularios (capítulo 5)
Código legible y nombres descriptivos	Parcial	Nombres descriptivos; falta unificar el estilo (p. ej. `BtnGuardar_Click` y `btnGuardar_Click`)

04 CAPÍTULO 4 Casos de uso

El único actor es el **bibliotecario**, quien opera el sistema. La aplicación no tiene inicio de sesión ni perfiles de usuario.

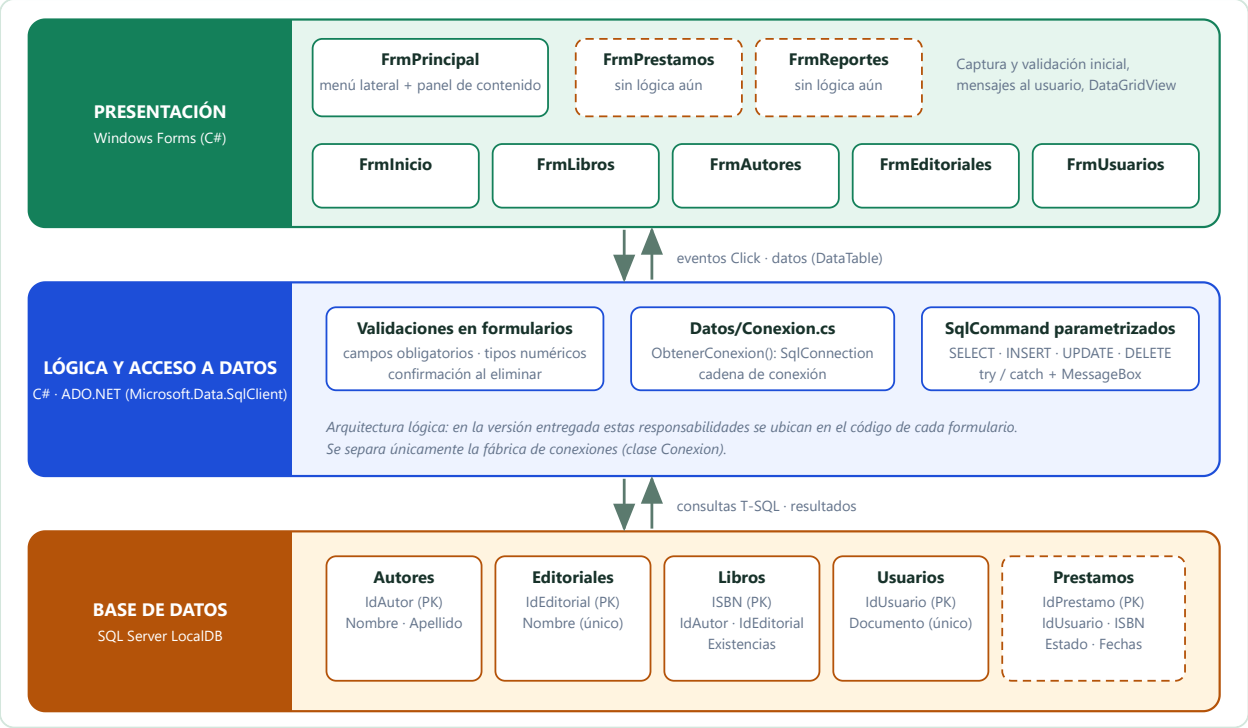


CASO DE USO	DESCRIPCIÓN	ESTADO
CU01 Gestionar libros	Registrar, listar, editar y eliminar libros, eligiendo autor y editorial de listas desplegables	Implementado
CU02 Gestionar autores	CRUD de autores; no se elimina un autor con libros asociados	Implementado
CU03 Gestionar editoriales	CRUD de editoriales; no se elimina una con libros asociados	Implementado
CU04 Gestionar usuarios	CRUD de usuarios; el documento es único	Implementado
CU05 Navegar entre módulos	Abrir cada módulo en el panel de contenido desde el menú lateral	Implementado
CU06 Buscar libro por ISBN	Cargar un libro en el formulario a partir de su ISBN (extiende CU01)	Implementado
CU07 Registrar préstamo	Asociar un usuario con un libro y registrar la fecha del préstamo	Pendiente
CU08 Registrar devolución	Marcar el préstamo como «Devuelto», registrar la fecha y sumar la existencia	Pendiente
CU09 Consultar reportes	Listados consolidados: préstamos activos, devueltos e historial	Pendiente

Flujo principal de CU01 (registrar un libro). (1) El bibliotecario abre *Libros*. (2) Diligencia ISBN, título, autor, editorial, categoría, año y existencias. (3) Pulsa *Guardar*. (4) El sistema valida campos obligatorios y valores numéricos. (5) Inserta el registro con una consulta parametrizada. (6) Muestra la confirmación y actualiza la cuadrícula. **Flujos alternos:** campos vacíos o no numéricos → advertencia sin guardar; ISBN repetido → error informado y sin inserción.

05 CAPÍTULO 5
Arquitectura del sistema

La aplicación se organiza en tres niveles lógicos. Como las instrucciones lo permiten, presentamos la estructura como una **arquitectura lógica** que corresponde a la organización real del código: el acceso a datos con `SqlCommand` y las validaciones se escriben dentro de cada formulario, y solo la creación de la conexión se separó en una clase propia.



CAPA	COMPONENTES	RESPONSABILIDAD
Presentación	`FrmPrincipal`, `FrmInicio`, `FrmLibros`, `FrmAutores`, `FrmEditoriales`, `FrmUsuarios`, `FrmPrestamos`, `FrmReportes`	Formularios, captura de datos, validación inicial y mensajes
Lógica y acceso a datos	Validaciones en los formularios, `Datos/Conexion.cs`, `SqlCommand` parametrizados	Reglas de validación, apertura de conexión y operaciones CRUD
Base de datos	SQL Server LocalDB, base `AcaBiblioteca`	Almacenamiento, relaciones e integridad

Tecnologías. C# sobre .NET 10 (`net10.0-windows`), Windows Forms, `Microsoft.Data.SqlClient` 7.1.1, `FontAwesome.Sharp` 6.6.0 (iconos del menú) y SQL Server LocalDB.

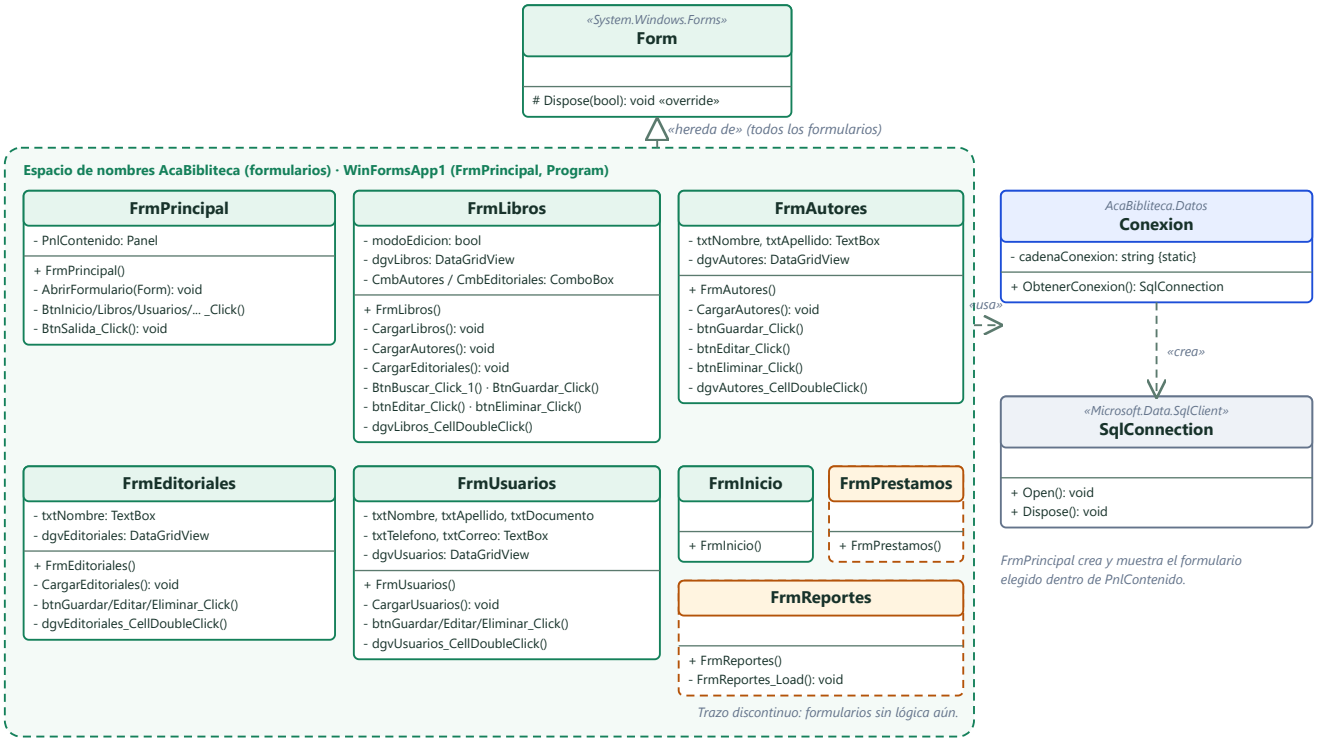
Estructura del repositorio.

```
ACA-Biblioteca/
├─ README.md                  instrucciones de ejecución
├─ Documentacion.md / .pdf    este documento
├─ Database/Script_AcaBiblioteca.sql
├─ docs/img/                  capturas y diagramas
└─ ACABiblioteca/
   ├─ ACABiblioteca.slnx      solución de Visual Studio
   └─ WinFormsApp1/
      ├─ Program.cs           punto de entrada → FrmPrincipal
      ├─ Datos/Conexion.cs    fábrica de conexiones
      └─ Frm*.cs / Frm*.Designer.cs / Frm*.resx
```

Flujo de navegación. `FrmPrincipal` contiene un panel (`PnlContenido`). El método `AbrirFormulario` vacía el panel, configura el formulario hijo como control (`TopLevel = false`, sin borde, acoplado) y lo muestra; así los módulos cambian sin abrir ventanas nuevas.

CAPÍTULO 6

06 Diagrama de clases y Programación Orientada a Objetos



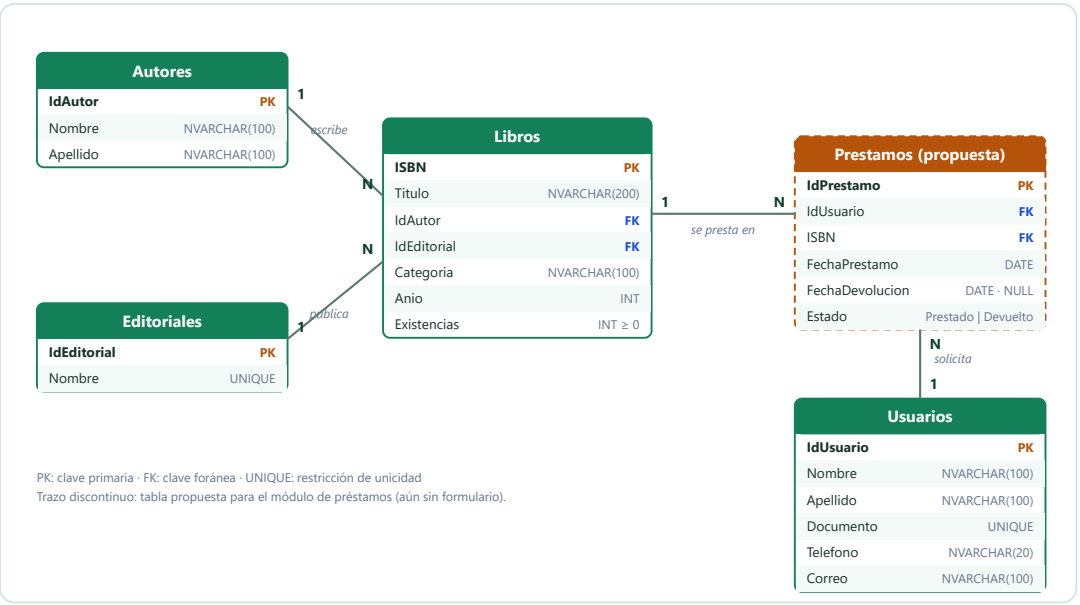
Conceptos de POO evidenciados

CONCEPTO	DÓNDE SE EVIDENCIA
Clases y objetos	Cada formulario es una clase (`FrmLibros` , `FrmAutores` ...); en tiempo de ejecución se crean objetos con `new FrmLibros()` , `new Conexion()` , `new SqlCommand(...)` .
Constructores	Cada formulario inicializa sus controles con `InitializeComponent()` y carga sus datos en el constructor (`FrmLibros()` llama a `CargarLibros()` , `CargarAutores()` y `CargarEditoriales()`).
Encapsulamiento	Campos y métodos `private` (`modoEdicion` , `CargarLibros` , `AbrirFormulario`); la cadena de conexión es `private static` y solo se expone mediante `ObtenerConexion()` .
Herencia	Todos los formularios heredan de `System.Windows.Forms.Form` .
Polimorfismo	Los formularios sobrescriben `Dispose(bool)` (`override`), y `FrmPrincipal.AbrirFormulario(Form)` acepta cualquier formulario hijo mediante su tipo base.
Métodos	Un método por responsabilidad: cargar datos, guardar, editar, eliminar, buscar y limpiar.
Colecciones	`DataTable` y `DataRow` para los datos, y las colecciones de filas de `DataGridView` .
Clases parciales	Cada formulario se divide en código propio (`.cs`) y código del diseñador (`.Designer.cs`).

Limitación reconocida. No definimos clases de entidad (`Libro` , `Autor` , `Usuario` ...) con propiedades propias: los datos viajan en `DataTable` y en los controles. Es la primera mejora que proponemos en las recomendaciones, pues permitiría separar por completo la lógica de negocio de la interfaz.

CAPÍTULO 7

07 Modelo entidad-relación y diccionario de datos



Relaciones. Un autor escribe muchos libros y una editorial publica muchos libros (1:N, ambas con clave foránea en `Libros`). Un usuario solicita muchos préstamos y un libro puede aparecer en muchos préstamos (1:N). Cada préstamo vincula exactamente un usuario con un libro. El script completo, con datos de prueba, está en `Database/Script_AcaBiblioteca.sql`.

Diccionario de datos

TABLA	CAMPO	TIPO	NULO	RESTRICCIÓN / DESCRIPCIÓN
Autores	IdAutor	INT IDENTITY	No	PK. Identificador del autor
	Nombre	NVARCHAR(100)	No	Nombre del autor
	Apellido	NVARCHAR(100)	No	Apellido del autor
Editoriales	IdEditorial	INT IDENTITY	No	PK. Identificador de la editorial
	Nombre	NVARCHAR(100)	No	UNIQUE. Nombre de la editorial
Usuarios	IdUsuario	INT IDENTITY	No	PK. Identificador del usuario
	Nombre, Apellido	NVARCHAR(100)	No	Datos personales
	Documento	NVARCHAR(20)	No	UNIQUE. Documento de identidad
	Telefono	NVARCHAR(20)	No	Teléfono de contacto
	Correo	NVARCHAR(100)	No	Correo electrónico
Libros	ISBN	NVARCHAR(20)	No	PK. Código del libro
	Titulo	NVARCHAR(200)	No	Título
	IdAutor	INT	No	FK → Autores.IdAutor
	IdEditorial	INT	No	FK → Editoriales.IdEditorial
	Categoria	NVARCHAR(100)	No	Categoría o género
	Anio	INT	No	CHECK entre 1000 y 2100
	Existencias	INT	No	CHECK ≥ 0; valor por defecto 0
Prestamos *(propuesta)*	IdPrestamo	INT IDENTITY	No	PK
	IdUsuario	INT	No	FK → Usuarios.IdUsuario
	ISBN	NVARCHAR(20)	No	FK → Libros.ISBN
	FechaPrestamo	DATE	No	Por defecto, fecha actual
	FechaDevolucion	DATE	Sí	Nula mientras no se devuelva; ≥ FechaPrestamo
	Estado	NVARCHAR(10)	No	`Prestado` o `Devuelto`, coherente con la fecha

CAPÍTULO 8

08 Conexión con SQL Server y validaciones

Conexión

La aplicación usa **SQL Server LocalDB** con autenticación integrada de Windows. La clase `Conexion` concentra la cadena de conexión y entrega una `SqlConnection` nueva a quien la solicita:

```
public class Conexion
{
    private static string cadenaConexion =
        "Server=(localdb)\\MSSQLLocalDB;Database=AcaBiblioteca;Trusted_Connection=True;";

    public SqlConnection ObtenerConexion()
    {
        return new SqlConnection(cadenaConexion);
    }
}
```

Cada operación abre su conexión dentro de un bloque `using`, de modo que se cierra siempre, incluso si ocurre un error. Ejemplo de inserción en `FrmAutores` (código abreviado):

```
using (SqlConnection cn = new Conexion().ObtenerConexion())
{
    cn.Open();
    string consulta = "INSERT INTO Autores (Nombre, Apellido) VALUES (@Nombre, @Apellido)";
    using (SqlCommand comando = new SqlCommand(consulta, cn))
    {
        comando.Parameters.AddWithValue("@Nombre", txtNombre.Text.Trim());
        comando.Parameters.AddWithValue("@Apellido", txtApellido.Text.Trim());
        comando.ExecuteNonQuery();
    }
}
```

Los valores viajan como **parámetros** y nunca se concatenan al texto SQL, lo que evita la inyección SQL. Las lecturas llenan un `DataTable` mediante `SqlDataAdapter` y lo asignan como `DataSource` del `DataGridView`.

Validaciones y manejo de excepciones

MÓDULO	VALIDACIONES EN LA INTERFAZ	VALIDACIONES EN LA BASE DE DATOS
Libros	Campos obligatorios; año y existencias deben ser enteros; autor y editorial seleccionados; ISBN para buscar	ISBN único (PK); FK a autor y editorial; año válido; existencias ≥ 0
Autores	Nombre y apellido obligatorios	FK desde Libros impide eliminar un autor con libros
Editoriales	Nombre obligatorio	Nombre único; FK desde Libros
Usuarios	Los cinco campos obligatorios	Documento único

Toda operación contra la base está dentro de `try/catch`; el error se muestra con un `MessageBox` y la aplicación continúa funcionando. Antes de eliminar, el sistema pide confirmación (*Sí/No*).

09 CAPÍTULO 9
Módulos desarrollados

Formulario principal (`FrmPrincipal`). Barra lateral con el logotipo y los botones Inicio, Libros, Usuarios, Autores, Editoriales, Préstamos, Reportes y Salir, y un panel de contenido donde se muestra cada módulo. La ventana se abre maximizada.

Inicio (`FrmInicio`). Mensaje de bienvenida que describe lo que se puede gestionar.

Libros (`FrmLibros`). Campos: ISBN, título, autor y editorial (listas desplegables cargadas desde la base), categoría, año y existencias. Botones: *Nuevo*, *Guardar*, *Editar*, *Eliminar* y *Cancelar*, y búsqueda por ISBN. Al hacer doble clic en una fila de la cuadrícula, sus datos pasan al formulario para editarlos. Las **existencias** del libro indican su disponibilidad.

Autores (`FrmAutores`), Editoriales (`FrmEditoriales`) y Usuarios (`FrmUsuarios`). Siguen el mismo patrón: campos de captura, cuadrícula con el listado completo y los botones *Nuevo*, *Guardar*, *Editar*, *Eliminar* y *Cancelar*. La edición y la eliminación actúan sobre la fila seleccionada. Autores y editoriales no se pueden eliminar si tienen libros asociados; los usuarios no pueden repetir el documento.

Préstamos (`FrmPrestamos`) y Reportes (`FrmReportes`). Los formularios existen y se abren desde el menú, pero aún no tienen controles ni lógica. **Diseño previsto:** *Préstamos* tendría listas desplegables de usuario y libro, selector de fecha (`DateTimePicker`), cuadrícula del historial y los botones *Prestar* y *Devolver*; al prestar, se registra el préstamo con estado «Prestado» y se descuenta una existencia, y al devolver se registra la fecha, el estado pasa a «Devuelto» y se suma la existencia, en una sola transacción. *Reportes* mostraría libros con autor y editorial, préstamos activos, préstamos devueltos e historial por usuario, mediante consultas con `JOIN`.

10 CAPÍTULO 10
Capturas de pantalla del sistema



Figura 1. Formulario principal e inicio

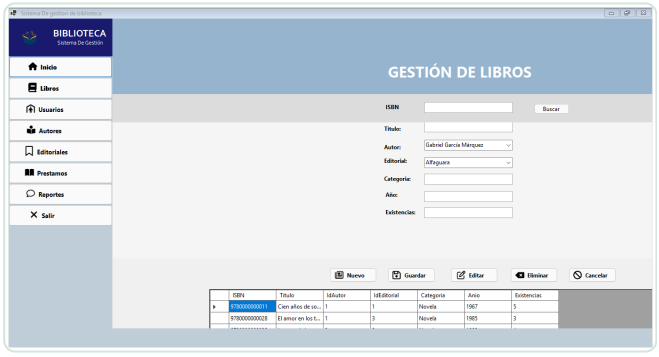


Figura 2. Módulo Libros con el listado



Figura 3. Libro cargado en el formulario al seleccionar una fila



Figura 4. Módulo Autores



Figura 5. Módulo Editoriales

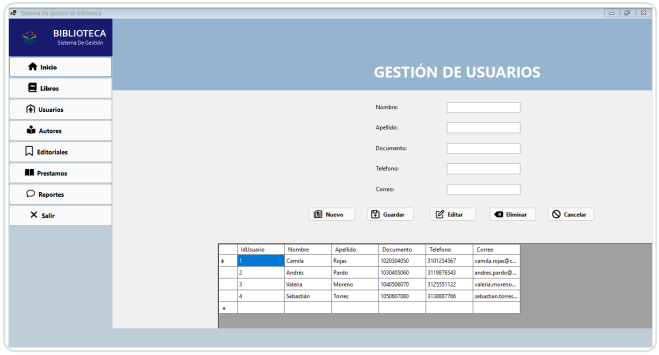


Figura 6. Módulo Usuarios

11 CAPÍTULO 11 Pruebas de funcionamiento

Ejecutamos las pruebas sobre la aplicación compilada, accionando los botones de cada formulario y verificando el mensaje mostrado y el contenido de la base `AcaBiblioteca` en SQL Server LocalDB, cargada con los datos de prueba del script (noviembre de 2026). **Resultado: 25 de 25 pruebas con el comportamiento esperado.**

ID	CASO DE PRUEBA	RESULTADO ESPERADO	RESULTADO OBTENIDO
A1	Autores: guardar con campos vacíos	Advertencia; no inserta	«Complete todos los campos antes de guardar.» Filas: 5 → 5 ✓
A2	Autores: guardar autor válido	Inserta y confirma	«Autor guardado correctamente.» Filas: 5 → 6 ✓
A3	Autores: editar nombre de un autor	Actualiza el registro	«Autor actualizado correctamente.» Cambio visible en BD ✓
A4	Autores: eliminar autor con libros	Rechaza por integridad	Error controlado: conflicto con `FK_Libros_Autores`; el registro se conserva ✓
A5	Autores: eliminar autor sin libros	Elimina y confirma	«Autor eliminado correctamente.» Filas: 6 → 5 ✓
E1	Editoriales: guardar con nombre vacío	Advertencia	«Ingrese el nombre de la editorial.» ✓
E2	Editoriales: guardar válida	Inserta y confirma	«Editorial guardada correctamente.» Filas: 4 → 5 ✓
E3	Editoriales: guardar nombre repetido	Rechaza duplicado	Error controlado: `UQ_Editoriales_Nombre`; sin inserción ✓
E4	Editoriales: eliminar con libros	Rechaza por integridad	Error controlado: `FK_Libros_Editorial` ✓
E5	Editoriales: editar nombre	Actualiza	«Editorial actualizada correctamente.» ✓
E6	Editoriales: eliminar sin libros	Elimina y confirma	«Editorial eliminada correctamente.» Filas: 5 → 4 ✓
U1	Usuarios: guardar con campos vacíos	Advertencia	«Complete todos los campos antes de guardar.» ✓
U2	Usuarios: guardar válido	Inserta y confirma	«Usuario guardado correctamente.» Filas: 4 → 5 ✓
U3	Usuarios: documento repetido	Rechaza duplicado	Error controlado: `UQ_Usuarios_Documento`; sin inserción ✓
U4	Usuarios: editar teléfono	Actualiza	«Usuario actualizado correctamente.» BD: 3149998877 ✓
U5	Usuarios: eliminar	Elimina y confirma	«Usuario eliminado correctamente.» Filas: 5 → 4 ✓
L1	Libros: guardar con campos vacíos	Advertencia	Mensaje de campos obligatorios; no inserta ✓
L2	Libros: año no numérico	Advertencia	«El año debe ser un número entero.» ✓
L3	Libros: guardar válido	Inserta y confirma	«Libro guardado correctamente.» Filas: 6 → 7 ✓

ID	CASO DE PRUEBA	RESULTADO ESPERADO	RESULTADO OBTENIDO
L4	Libros: ISBN repetido	Rechaza duplicado	Error controlado: `PK_Libros`; sin inserción ✓
L5	Libros: editar existencias (3 → 7)	Actualiza	«Libro actualizado correctamente.» BD: 7 ✓
L6	Libros: buscar ISBN existente	Carga el libro	«Libro encontrado...»; título «Cien años de soledad» cargado ✓
L7	Libros: buscar ISBN inexistente	Informa	«No se encontro ningun libro con el ISBN proporcionado.» ✓
L8	Libros: buscar con ISBN vacío	Advertencia	«Por favor, Ingrese un ISBN para buscar.» ✓
L9	Libros: eliminar libro	Elimina y confirma	«Libro eliminado correctamente.» Filas: 7 → 6 ✓

Las pruebas de **préstamos, devoluciones y reportes** no se ejecutaron porque esos módulos están pendientes.

12 CAPÍTULO 12 Control de versiones con Git y GitHub

El proyecto está en el repositorio público **https://github.com/PORTHALTHEMANIACA51/ACA-Biblioteca** (rama `main`). Trabajamos con Git desde la creación del proyecto; el historial incluye, entre otros, los siguientes commits:

COMMIT	FECHA	DESCRIPCIÓN
`7739dab`	26/10/2026	Initial commit: creación del repositorio
`38693ee`	27/10/2026	Creación de la documentación
`d7cb468`	09/11/2026	Primeras instrucciones para visualizar el código
`d668f41`	09/11/2026	Actualización de los formularios
`d7bf52e`, `4fe6034`	09/11/2026	Corrección del módulo de Autores
`72599d1`	09/11/2026	Modificación del diseño (Inicio, Libros, Editoriales)
`ac044a9`	09/11/2026	Instrucciones para visualizar el código

El repositorio reúne el código fuente de la solución, el script SQL (`Database/`), el `README.md` con las instrucciones de ejecución, las capturas y diagramas (`docs/img/`) y esta documentación. Para ejecutar el proyecto: (1) instalar Visual Studio con la carga de trabajo de escritorio .NET y SQL Server LocalDB; (2) ejecutar el script en `(localdb)\MSSQLLocalDB`; (3) abrir `ACABiblioteca.slnx` y ejecutar con F5.

Limitaciones conocidas y trabajo futuro

- Módulos de **Préstamos, Devoluciones y Reportes** sin implementar (RF05, RF06 y RF07).
- La búsqueda de libros es solo por ISBN; no hay búsqueda por título ni por autor.
- No se valida el formato del correo electrónico ni se limita la longitud de los campos antes de enviarlos a la base.
- Algunos mensajes de error de la base se muestran en inglés, tal como los devuelve SQL Server.
- La cadena de conexión está fija en el código.

13 CAPÍTULO 13 Conclusiones y recomendaciones

Conclusiones

1. Logramos una aplicación de escritorio funcional que administra libros, autores, editoriales y usuarios sobre SQL Server, con operaciones CRUD completas y las 25 pruebas funcionales realizadas con el resultado esperado.

2. La integridad de los datos se garantiza en dos niveles: validaciones en la interfaz y restricciones en la base de datos (claves foráneas, unicidad y `CHECK`). Las pruebas A4, E4, E3, U3 y L4 muestran que un error de integridad se informa sin detener la aplicación.
3. El uso de consultas parametrizadas, bloques `using` y `try/catch` hizo que el acceso a datos fuera seguro y estable.
4. Aplicamos conceptos de POO (clases, constructores, encapsulamiento, herencia y polimorfismo por sobrescritura), aunque la lógica de negocio quedó dentro de los formularios.
5. Cumplimos los módulos de catálogos y la navegación; los módulos de préstamos, devoluciones y reportes quedaron con su estructura y su diseño de datos definidos, pero sin implementar. Preferimos reportarlo así antes que presentarlo como terminado.
6. El uso de Git y GitHub nos permitió compartir el código entre los dos y conservar el historial de cambios.

Recomendaciones

1. **Completar** Préstamos, Devoluciones y Reportes con una transacción que registre el préstamo y descuente la existencia de forma atómica.
2. **Crear clases de entidad** (`Libro`, `Autor`, `Editorial`, `Usuario`, `Prestamo`) y una capa de acceso a datos (repositorios) para sacar el SQL de los formularios.
3. **Añadir validaciones** de correo electrónico, longitudes y rangos, y traducir los mensajes de error de la base a mensajes propios.
4. Leer la cadena de conexión desde un archivo de configuración.
5. Incluir un archivo `.gitignore` y no versionar las carpetas `bin`, `obj` y `.vs`.
6. Corregir los errores ortográficos de algunos mensajes y unificar el estilo de nombres.

CAPÍTULO 14

14 Referencias bibliográficas

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Microsoft. (s. f.). *Documentación de Windows Forms*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/

Microsoft. (s. f.). *Documentación de C#*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/csharp/

Microsoft. (s. f.). *Espacio de nombres Microsoft.Data.SqlClient*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/api/microsoft.data.sqlclient

Microsoft. (s. f.). *SQL Server Express LocalDB*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/database-engine/configure-windows/sql-server-express-localdb

OWASP Foundation. (s. f.). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

GitHub. (s. f.). *Documentación de GitHub*. https://docs.github.com/es

PROGRAMACIÓN AVANZADA

Sistema de Gestión de Biblioteca

Documento técnico del proyecto final

Juan Romero · John David Montilla

Ficha 54465 · Docente: Verónica Castro Munar · 2026

`github.com/PORTHALTHEMANIACA51/ACA-Biblioteca`